SSOT Containment Directive

Origin architecture cleanup guidance for implementation agents

Date	May 24, 2026
Status	Implementation directive
Scope	Rein in single-source-of-truth violations before further origin-system expansion

Core verdict: this is a containment problem before it is a planning problem. The implementation agent should not continue expanding origin tracking, debug views, fact schemas, hash policies, or export models until duplicated sources of truth are identified, deleted, demoted, or fenced by tests.

Non-negotiable instruction to the agent

Stop expanding the design. Do not add new origin systems, registries, export models, fact schemas, debug mappings, hash policies, or documentation-only abstractions until the SSOT violations are inventoried and corrected.

First produce a concrete SSOT audit with file paths and line references. For each duplicated concept, identify the single authoritative owner, every competing representation, what gets deleted, what becomes derived, and what tests prevent recurrence.

Contents

1. Executive summary

The project does not need one grand origin object shared by every system. It needs a disciplined chain of ownership: local, typed identity inside each compiler phase; explicit origin links between phases; stable projections at export boundaries; and derived consumers for facts, debug data, hashes, and reports.

The main failure mode to avoid is solving SSOT violations by adding another abstraction layer. That usually creates a new authority rather than removing the duplicate one.

The next useful artifact from the agent is not a broad roadmap. It is a patch-backed audit: where are the duplicate truths, which one owns the truth, and which duplicates were deleted or demoted?

2. Authority model

The target architecture should use this ownership rule:

```
Source syntax owns syntax identity.
HIR owns HIR identity.
MIR owns MIR identity.
Codegen owns backend identity.
Bytecode owns PC/range identity.

Origin links connect phase-owned nodes.
Stable projections export them.
Analysis layers consume them.
```

Decision rules

- Phase crates own phase-local typed origin identity.
- common may provide generic identity machinery, but not semantic cross-phase ownership.
- Export, debug, fact, and hash layers consume stable projections. They do not define the authority for identity.
- Facts, debug records, reports, and hashes are derived views. They must not become second sources of truth.
- Documentation states invariants, but code and tests enforce them. A document alone is not SSOT.

Healthy split

Layer	Owns	Should not own
common	Generic typed identity machinery and stable-key primitives.	HIR/MIR/codegen semantic variants or a global compiler-universe enum.
HIR crate	HIR expression, statement, body, and owner-scoped origins.	MIR, bytecode, debug, or fact-table identity.
MIR crate	MIR statement, terminator, block, runtime-instance, and owner-scoped origins.	Serialized external identity or source-debug export truth.
Codegen/backend crate	Backend function, instruction, layout, and bytecode PC/range origins.	Source-level authority or pre-optimization truth for post-optimization artifacts.
Export/debug/fact/hash layers	Derived projections and views for external consumption.	The canonical identity of compiler nodes.

3. High-risk SSOT violations

Multiple origin identity systems

Watch for: Bad signs: HirExprOrigin, HirStmtOrigin, OriginNode, ProvenanceNodeId, SourceNode, AnalysisNode, FactNode, and DebugNode all try to name the same underlying source relationship.

Required correction: Correct rule: internal identity is owned by the phase; external identity is a projection.

common as a semantic dumping ground

Watch for: Bad signs: common contains a concrete global enum with HIR, MIR, Sonatina, bytecode, debug, and fact variants.

Required correction: Correct rule: common provides the identity mechanism; phase crates define the meaning.

Fact, debug, and hash layers acting as authorities

Watch for: Bad signs: MIR origin says one thing, fact graph says another, debug map says another, and the hash graph silently invents IDs again.

Required correction: Correct rule: analysis products reference established origin keys; they do not create canonical identity.

Stable/export IDs confused with internal IDs

Watch for: Bad signs: Body, RuntimeInstance, InstId, BlockId, StatementId, FuncRef, or PC range values are serialized directly as stable origin truth.

Required correction: Correct rule: internal typed key -> stable projection -> external consumers.

Documentation as fake SSOT

Watch for: Bad signs: architecture docs describe invariants that the code does not encode and tests do not enforce.

Required correction: Correct rule: every invariant should have a code owner and a regression test.

4. Required cleanup sequence

1. Inventory

Build a file-and-line audit of duplicated concepts before making more design changes.

2. Delete or demote duplicates

Every competing representation must be deleted, renamed as derived, moved to the owning phase crate, made private, or converted into a stable projection only.

3. Add compile-time fences

Prefer type-level and compile-fail protection over runtime-only checks. The wrong origin type should fail to compile in the wrong context.

4. Add thin conversion boundaries

Create explicit export/projection functions where internal origin identity becomes stable external identity. Avoid casual conversions when database, package, or source-map context is required.

5. Resume implementation only after the SSOT map passes

MIR propagation, hashing, facts, debug exports, and optimizer provenance should proceed only after identity authority is clean.

5. SSOT audit template

The agent should fill a table like this before proposing another implementation phase:

Concept	Current representations	Intended SSOT	Action	Regression test
HIR expr origin	List exact types, files, and lines.	HIR crate expr-origin type.	Keep one; delete or demote others.	Stmt origin cannot be used as expr origin. Same local ID in two bodies stays distinct.
HIR stmt origin	List exact types, files, and lines.	HIR crate stmt-origin type.	Keep one; delete or demote others.	Expr origin cannot be used as stmt origin. Owner plus local ID required.
MIR stmt origin	List exact types, files, and lines.	MIR crate stmt-origin type.	Keep one; require owner, block, and statement context.	No raw statement ID crosses the MIR boundary.
Bytecode origin	List exact types, files, and lines.	Backend-owned PC/range origin.	Keep post-layout authority; derive views.	PC ranges map through post-opt/layout origins.
Stable export key	List serializers, fact IDs, debug IDs, and report IDs.	Export-layer stable key.	Make internal IDs non-serializable as stable truth.	Stable key roundtrips. Raw backend IDs cannot be exported directly.
Hash node identity	List graph/hash node IDs and labels.	Derived from phase-owned origin or stable projection.	Remove independent canonical node ID.	Node-content changes affect digest. Graph edges do not suppress child content.

6. Acceptance criteria

The cleanup is complete only when the following are true:

- There is exactly one authoritative owner for each origin identity concept.
- Every competing representation is deleted, demoted to a derived view, made private, or moved to the correct owner.

- common does not model the semantic universe of all compiler phases.
- Fact, debug, report, and hash systems consume existing origin keys or stable projections instead of inventing canonical IDs.
- No raw owner-scoped local ID escapes without its owner context.
- No internal handle is serialized or documented as an external stable identity unless explicitly projected through an export boundary.
- The code has tests that fail when the SSOT rule is violated.
- The implementation diff removes more duplicated authority than it adds.

Compile-time fences to prefer

- A HIR statement origin cannot be passed where a HIR expression origin is expected.
- An OriginKey cannot be constructed from only a local ID.
- A raw backend instruction ID cannot be exported as stable origin identity.
- Hash/fact/debug systems cannot create canonical phase-node IDs without referencing the authoritative phase-owned origin.

7. Agent response format

Require the agent to respond in this structure:

```
1. SSOT audit table
   - Concept
   - Current representations with file:line references
   - Intended authoritative owner
   - Delete/demote/move/private/projection action
   - Regression test

2. Minimal cleanup patch
   - Files changed
   - Duplicates removed
   - Boundaries made explicit

3. Test evidence
   - Compile-fail or type-level tests where possible
   - Runtime regression tests where necessary
   - Any tests not run, with a clear reason

4. Remaining risks
   - Only risks that remain after the cleanup patch, not a new roadmap
```

Refusal condition

If the agent proposes new architecture, new registries, new enums, new export schemas, new Datalog schemas, new debug maps, or new hash policies before completing the audit, treat that as scope creep. Redirect it to the audit and cleanup patch.

8. One-page checklist

Check	Pass condition
Stop expansion	No new systems until SSOT audit and cleanup patch exist.
Name the owner	Every identity concept has one authoritative crate/layer.
Delete or demote	Duplicates are removed or explicitly marked derived.
Keep identity local	Phase-owned internal identity remains typed and owner-scoped.
Use explicit links	Cross-phase relationships are links, not a universal ID namespace.
Project at the boundary	Stable/export/debug/fact IDs are derived from internal keys.
Fence with tests	Wrong-origin usage fails at compile time where practical.
Make docs subordinate	Docs describe invariants already enforced by code/tests.

Architectural stance

Make identity boring, local, typed, and owned.

Make cross-phase relationships explicit links.

Make exports derived.

Make tests enforce the boundaries.

Do not let another abstraction become another source of truth.